

Программирование в лингвистике

Структурирование данных и оптимизация кода

Измерение сложности алгоритма и памяти.

Структуры данных: какую выбрать, чтобы было удобно?

`namedtuple`, `deque` из `collections`

Инструменты `itertools`

Выбор структуры данных и читаемость

```
data = [  
    [101, "Иванов", ["смартфон", "чехол"], True],  
    [102, "Петрова", ["книга", "ручка"], False],  
    [103, "Сидоров", ["кофемашина"], True]  
]  
  
def process_undelivered_orders(data):  
    for order in data:  
        if not order[3]:  
            print("Отправить уведомление клиенту", \  
                  order[1], "по заказу №", order[0])  
            order[2].append("подарочный блокнот")  
  
process_undelivered_orders(data)
```

- ▶ Почему программу **долго писать**?
- ▶ Потому что код плох с точки зрения **читаемости**.
- ▶ Данные хранятся во вложенных списках, где легко **запутаться в индексах** data[0][2].
- ▶ Что лучше было использовать?
- ▶ Правильный выбор структуры данных **экономит 80% усилий** при программировании.

Выбор структуры данных и архитектура

```
data = [  
    [101, "Иванов", ["смартфон", "чехол"], True],  
    [102, "Петрова", ["книга", "ручка"], False],  
    [103, "Сидоров", ["кофемашина"], True]  
]  
  
def process_undelivered_orders(data):  
    for order in data:  
        if not order[3]:  
            print("Отправить уведомление клиенту", \  
                  order[1], "по заказу №", order[0])  
            order[2].append("подарочный блокнот")  
  
process_undelivered_orders(data)
```

- ▶ Почему программу **тяжело поддерживать**?
- ▶ Потому что код плох с точки зрения **архитектуры**.
- ▶ Если завтра пользователь попросит добавить поле даты заказа в начало списка, все **придется переписывать**.
- ▶ Что лучше было использовать?
- ▶ Правильный выбор структуры данных критически важен для **возможности обновления**.

Выбор структуры данных и ресурсы

```
data = [  
    [101, "Иванов", ["смартфон", "чехол"], True],  
    [102, "Петрова", ["книга", "ручка"], False],  
    [103, "Сидоров", ["кофемашина"], True]  
]  
  
def process_undelivered_orders(data):  
    for order in data:  
        if not order[3]:  
            print("Отправить уведомление клиенту", \  
                  order[1], "по заказу №", order[0])  
            order[2].append("подарочный блокнот")  
  
process_undelivered_orders(data)
```

- ▶ Почему программа **плохо масштабируется**?
- ▶ Потому что код плох с точки зрения **затрат памяти** и **быстродействия**.
- ▶ При миллионе заказов тратим много ресурсов на поддержку **изменяемых** подписков.
- ▶ Что лучше было использовать?
- ▶ Правильный выбор структуры данных критически важен для **масштабируемости**.

Преобразуем списки в named tuple

```
from collections import namedtuple

Order = namedtuple('Order', ['id', 'client_name', 'items',
                              'is_delivered'])

data = [
    Order(101, "Иванов", ["смартфон", "чехол"], True),
    Order(102, "Петрова", ["книга", "ручка"], False)
]

def process_undelivered_orders(data):
    for order in data:
        if not order.is_delivered:
            print("Отправить уведомление клиенту", \
                  order[1], "по заказу №", order[0])
            order[2].append("подарочный блокнот")
```

- ▶ **named tuple** из **collections** — функция для создания подклассов стандартного кортежа с доступом к элементам как по индексу, так и по имени.
- ▶ Данные защищены от случайных изменений после создания.
- ▶ Объекты **весят мало**, как и обычные кортежи, и так же **быстро обрабатываются**.
- ▶ Другой вариант — **dataclass** из **dataclasses**.

Быстродействие и сложность алгоритма

- ▶ Раньше мы измеряли скорость работы программы **абсолютно** (в миллисекундах), но могли это сделать **только после** ее написания.
- ▶ **Big O (О-нотация)** позволяет оценить время работы **предсказательно**, в зависимости от того, **с каким количеством данных и как** мы будем работать.
- ▶ Для О-нотации важно не точное количество миллисекунд, а как быстро растет **количество операций** – в худшем (лучшем) случае – при росте объёма данных.
- ▶ Эта относительная оценка называется **вычислительной сложностью** алгоритма.
- ▶ Если сложность **линейная** – обозначается **$O(n)$** , – это **нормально**, но в случае большого объема данных программе потребуется много времени.
- ▶ Лучше **$O(\ln n)$** или **$O(\log n)$** – **логарифмическая** сложность (на больших объемах данных время растет медленнее), еще лучше **$O(1)$** – **константное время**!
- ▶ Хуже: **$O(n \log n)$** , **$O(n^2)$** , **$O(n^3)$** , **$O(\exp(n))$** , **$O(n!)$** .

Примеры алгоритмов по сложности

- ▶ $O(1)$ – проверка делимости числа.
- ▶ $O(n)$ – поиск наименьшего или наибольшего элемента в несортированном списке.
- ▶ $O(n \log n)$ – самая быстрая сортировка сравнением (**Tree sort** оптимизированный).
- ▶ $O(n^2)$ – сортировка пузырьком (**Bubble sort**).
- ▶ $O(n^3)$ – обычное умножение двух $n \times n$ матриц.
- ▶ $O(\exp(n))$ - решение задачи коммивояжёра с помощью перебора всех подмножеств.
- ▶ $O(n!)$ - решение задачи коммивояжёра методом полного перебора.

Алгоритмы с **полиномиальным временем** – класс алгоритмов, сложность которых выражима как многочлен вида $a * n^k + b * n^{(k-1)} + \dots$, где $k, a, b, \dots$ - константы. В конкретных случаях пишем сложность по максимальной степени.

Оценка сложности на примере очереди*

```
n = 10 # Алгоритм для n=10
my_list = [] # Создаем список

for i in range(n): # Генерируем новый элемент
    my_list.insert(0, i) # Вставляем слева со сдвигом
```

Ср. с альтернативным алгоритмом через deque:
`deque` из `collections` (двусторонняя очередь) – структура данных, устроенная так, что добавление элемента в самое начало списка не будет сдвигать все остальные элементы, а просто построит очередь слева.

- ▶ Оценим сложность программы, которая добавляет элементы в начало списка длиной n в цикле.
- ▶ Метод `.insert()` проходит по всему списку и переставляет элементы на $+1$ позицию.
- ▶ Т.е. он имеет сложность $O(n)$.
- ▶ Использование цикла `for` еще умножает сложность на n .
- ▶ В итоге получаем $O(n^2)$.

* Не путаем очередь как структуру данных с очередью мультипроцессинге.

Оптимизация с использованием deque

```
from collections import deque
```

```
n = 10 # Алгоритм для n=10
```

```
my_deque = deque() # Создаем очередь
```

```
for i in range(n): # Генерируем новый элемент  
    my_deque.appendleft(i) # Добраиваем очередь
```

- ▶ Оценим сложность алгоритма через `deque`.
- ▶ Метод `.appendleft` имеет $O(1)$ сложность.
- ▶ Использование цикла `for` умножает сложность на n .
- ▶ В итоге имеем $O(n)$ сложность.

Выбор структуры данных для разных конкретных задач зависит от разных факторов, между которыми важно соблюсти баланс: так, «платой» за быстроту `deque` будет большой объем памяти на ее хранение, т.к. каждый элемент должен хранить не только значение, но и ссылки на предыдущий и следующий элементы.

Оценка сложности на примере пузырька

```
a = [0] * 10 # Алгоритм для n=10
for i in range(10): # Заполнение исходного списка
    a[i] = (int(input("Введите число: ")))

for i in range(9): # Проходы для n-1 позиций
    for j in range(9-i): # Просмотр начала списка
        if a[j] > a[j+1]: # Сравнение пар элементов
            a[j], a[j+1] = a[j+1], a[j] # Всплытие больших
```

Алгоритм сортировки пузырьком

1. Просматриваем список.
2. Для **каждой пары** соседних элементов выполняем:
если первый больше, чем второй, **меняем их местами**.
3. Повторяем такие проходы, **пока** список **не упорядочен**.

- ▶ Сортировка пузырьком на списке из n элементов за первый проход выполняет $n-1$ операцию сравнения, за второй – $n-2$ операции, и так далее.
- ▶ В худшем случае (алгоритм не оптимизирован или изначальный список дан в обратном порядке) делаем $n-1$ проход.
- ▶ Суммирование дает $n^2/2$.
- ▶ В итоге получаем как верхнюю оценку $O(n^2)$ сложность.
- ▶ **Как оптимизировать?**

Рекурсия: красота требует жертв

- ▶ Некоторые задачи в программировании удобно решать с помощью создания **рекурсивной функции** — функции, **вызывающей саму себя** с другими параметрами, приближающими нас к **базовому** (нерекурсивному) **случаю**.
- ▶ Такой код обычно лаконичен и выглядит **красиво**.
- ▶ Но за эту красоту есть **цена** — рекурсия потребляет очень много **ресурсов**.
- ▶ Когда функция вызывает саму себя, она не заменяет старую функцию. Компьютер вынужден **«заморозить»** текущее вычисление, т.е. положить его в специальную структуру оперативной памяти (**стэк**) и открыть новую копию.
- ▶ Таким образом память забивается очень быстро, и вдобавок зачастую при этом делается много **дублирующей работы**. Результат — **программа зависает**.
- ▶ При этом если решать ту же задачу **другим методом**, она зачастую может быть решена **достаточно экономно**.

Рекурсия: пример чисел Фибоначчи

Формула для n-го числа Фибоначчи:

$$F_n = F_{n-1} + F_{n-2}$$

```
def fib_recursive(n): # НЕ делайте так!!!  
    if n <= 1:  
        return n  
    return fib_recursive(n - 1) + fib_recursive(n - 2)
```

```
def fib_iterative(n): # А так – делайте.  
    a, b = 0, 1  
    for i in range(n):  
        a, b = b, a + b  
    return a
```

- ▶ Рекурсивный алгоритм вычисления последовательности Фибоначчи очень короткий, но ужасно неэффективный.
- ▶ По времени – $O(\exp(n))$: чтобы посчитать `fib_recursive(40)`, функция сделает больше триллиона вызовов!
- ▶ По памяти – $O(n)$: стек вызовов будет расти пропорционально n .
- ▶ Если передать `n = 1000`, Python просто принудительно уронит программу с ошибкой `RecursionError`.
- ▶ При этом обычный цикл решает обе проблемы.

Функции-генераторы и yield

- ▶ `yield` – ключевое слово в Python, которое превращает обычную функцию в генератор.
- ▶ Генераторы – функции, использующие парадигму ленивых вычислений.
- ▶ Мы знаем генераторные выражения для списков, словарей и множеств.
- ▶ Они основаны на цикле `for`, который содержит инструкции по заполнению коллекции.
- ▶ Но не все генераторные выражения, основанные на цикле `for`, ленивые.

```
lst = [x**2 for x in range(5)]  
print(lst) # Выдаст готовый список
```

```
[0, 1, 4, 9, 16]
```

Vs

```
gen = (x**2 for x in range(5))  
print(gen) # Выдаст объект генератора
```

```
<generator object <genexpr> at 0x...>
```

Синтаксис функции-генератора

```
def gen_func():  
    for x in range(5):  
        yield x**2  
  
my_gen = gen_func()  
print(my_gen) # Выдаст объект генератора
```

`yield` заменяет `return` в функциях-генераторах:

- вместо того, чтобы вернуть объект и уничтожить все данные о работе функции (пространство имен и т.д.), `yield` замораживает функцию до следующего вызова.
- когда будет `next` или следующая итерация `for`, функция продолжит работу с того же места.

- ▶ Эта функция делает то же, что и генераторное выражение на предыдущем слайде, но она имеет **имя** и задана через **def**.
- ▶ Генераторное выражение — это **как лямбда-функция** среди генераторов: не сохраняется в памяти, не может быть вызвано повторно и создается для простых задач.
- ▶ Функция-генератор — это полноценная функция.

* **Не путаем** генераторное выражение, функцию-генератор и объект генератора.

Оптимизация через функцию-генератор

```
def read_lazy(filename):  
    with open(filename, 'r', encoding='utf-8') as f:  
        for line in f:  
            clean_line = line.strip()  
            yield clean_line  
  
file_generator = read_lazy("f.txt")  
line = next(file_generator) # Считываем одну строку  
print(line)  
  
for line in file_generator: # Считываем строки в цикле  
    if "Бинго" in line:  
        print("Нашли нужную строку!")  
        break
```

- ▶ Эта функция лениво считывает и очищает строки в файле.
- ▶ Она никогда не загружает файл целиком в память.
- ▶ Одна строка читается, обрабатывается и выдается.
- ▶ Легкая и быстрая.
- ▶ Но файл остается открытым всё то время, пока мы вызываем `next()` или крутим генератор в цикле.
- ▶ `with` не закроет файл, пока генератор не отработал.

Генераторы модуля `itertools`

- ▶ Модуль `itertools` – это встроенный в Python инструмент для создания итераторов.
 - ▶ Все функции в этом модуле работают по принципу **ленивых вычислений**.
 - ▶ Они **генерируют элементы «на лету»**, а не создают в памяти новые списки.
 - ▶ Это позволяет обрабатывать гигантские (и даже бесконечные) объемы данных с **минимальными затратами ресурсов**.
 - ▶ Важнейшие инструменты:
 - `itertools.chain(list1, list2)` – сшивает два списка «на лету», начиная перебирать второй только тогда, когда в первом закончатся элементы;
 - `itertools.islice(gen, start, stop, step)` – создает срезы для генераторов;
 - `itertools.groupby(iterable, key)` – группирует элементы итератора по признаку;
- ВНИМАНИЕ: работает корректно только с отсортированными коллекциями.

Спасибо за внимание!